

Today Todo - 간단한 할 일 앱

Flutter와 Riverpod으로 만든 Windows 네이티브 To Do 앱입니다. 목록·추가·삭제·완료 체크를 제공하며, 할 일은 메모리에 보관합니다. 앱을 종료하거나 재시작하면 목록이 초기화됩니다.

1. 실행하기

Windows에서 소스를 빌드해 실행합니다. 앱 실행.cmd는 로컬에 준비된

output/windows-simple/Release/today_todo.exe를 실행하는 보조 스크립트입니다.

실행 파일을 배포할 때는 같은 폴더의 DLL과 data도 함께 포함해야 합니다.

GitHub에서 받은 소스 또는 제출 ZIP에는 빌드 결과가 포함되지 않으므로 Flutter와 Windows 빌드 도구를 준비한 뒤 프로젝트 폴더의 PowerShell에서 실행합니다.

```
powershell -ExecutionPolicy Bypass -File .\scripts\run_app.ps1
```

debug 앱이 열립니다. 할 일을 입력하고 추가하거나 Enter를 누릅니다. 체크박스로 완료 상태를 바꾸고 휴지통 버튼으로 삭제합니다. 실행 터미널에서 q를 누르면 종료됩니다.

```
powershell -ExecutionPolicy Bypass -File .\scripts\run_app.ps1 -Profile
```

```
powershell -ExecutionPolicy Bypass -File .\scripts\run_app.ps1 -Build
```

-Profile은 DevTools 성능 관찰용 실행이고, -Build는 release 빌드입니다. 빌드 후 실행 파일은 다음 위치에 생깁니다.

```
.\build\windows\x64\runner\Release\today_todo.exe
```

빌드를 마치면 앱 실행.cmd로 실행할 수 있습니다. 실행 파일을 다른 폴더에 옮길 때에는 DLL·data가 들어 있는 **Release 폴더 전체**를 복사합니다. 실행 스크립트는 프로젝트의 .tooling/flutter SDK 또는 PATH의 Flutter를 사용합니다.

-ExecutionPolicy Bypass는 해당 PowerShell 프로세스에만 적용됩니다.

2. 기능과 과제 범위

요구사항	구현
기본 기능 20점	할 일 목록 표시, 제목 추가, 고유 ID로 삭제
Riverpod 보너스 10점	같은 앱의 목록과 완료 상태를 NotifierProvider로 관리
네이티브 DevTools 보너스 10점	Windows 앱의 Inspector·Timeline·Memory·Performance 관찰
추가 기능	완료 체크 및 완료 취소
제출물	자체 코드, 실행 명령·구조·보너스·스크린샷을 포함한 Readme.pdf

공백만 있는 제목은 추가하지 않습니다. 제목 앞뒤 공백을 정리하며 같은 제목의 여러 항목은 서로 다른 ID로 구분합니다. 완료한 항목도 목록에 남아 완료 표시를 확인하거나 삭제할 수 있습니다. 빈 목록에는 안내를 표시합니다.

과제 마감은 **2026-10-11 일요일 23:59 KST**입니다.

3. 개발 환경과 일반 명령

확인한 환경은 Flutter 3.47.6 stable, Dart 3.13.5, flutter_riverpod 3.4.3, DevTools 2.60.0, Windows x64입니다. 패키지 버전은 pubspec.lock으로 고정합니다.

Flutter SDK를 설치하고 bin 폴더를 PATH에 등록합니다. Windows 네이티브 빌드를 위해 Visual Studio 또는 Build Tools에 **Desktop development with C++** 작업과 Windows SDK를 설치합니다.

PATH에 Flutter가 있는 일반 환경에서는 다음 명령을 사용합니다.

```
flutter --version
flutter doctor -v
flutter pub get
dart format lib test test_driver scripts
flutter analyze
flutter test --reporter expanded
flutter run -d windows
flutter run -d windows --profile
flutter build windows
```

debug와 profile 실행은 하나씩 종료한 뒤 다음 명령을 실행합니다. `flutter pub get`은 의존성을 설치하며, SDK·패키지 소스와 생성 캐시는 코드 제출물에서 제외합니다.

4. 코드 구조와 위젯

파일	역할
lib/main.dart	ProviderScope 안에서 MaterialApp 시작
lib/models/todo.dart	Todo의 ID·제목·완료 상태 정의
lib/providers/todo_provider.dart	목록 상태 및 추가·삭제·완료 변경
lib/screens/todo_screen.dart	입력 폼·목록·빈 목록 안내
lib/widgets/todo_tile.dart	개별 항목의 체크박스·제목·삭제 버튼
test/	상태 변경과 화면 동작 검사
test_driver/, scripts/	네이티브 촬영·검증·실행·PDF·제출 도구

화면은 `ConsumerStatefulWidget`으로 구현하여 입력창의 `TextEditingController`와 `FocusNode`를 소유하고 `dispose`에서 정리합니다. `ScaffoldAppBar` 아래에 `TextField`와 `FilledButton.icon`, 전체 개수, `Expanded` 안의 `ListView.builder`를 둡니다. `TodoTile`은 `ListTile`의 제목·`Checkbox`·삭제 `IconButton`으로 구성하며 완료 제목에는 취소선을 표시합니다.

빈 제목은 입력창의 **할 일을 입력하세요**. 오류로 안내합니다. 새 입력 시 오류를 해제하고 정상 추가하면 입력창을 비운 뒤 포커스를 유지합니다. Enter도 같은 추가 처리를 호출합니다.

`ref.watch(todoProvider)`로 현재 목록을 읽어 상태 변경 시 화면을 다시 구성합니다. 버튼 동작은 `ref.read(todoProvider.notifier)`로 변경 메서드를 호출합니다. 제목 입력은 화면의 임시 상태이고 할 일 목록은 Riverpod 상태입니다.

5. 상태 자료구조와 변경 흐름

Todo는 `int id`, `String title`, `bool isCompleted`를 가진 불변 모델입니다. `copyWith({bool? isCompleted})`로 ID·제목을 유지한 새 객체를 만듭니다. Riverpod 상태는 `List<Todo>`이며 `NotifierProvider<TodoNotifier, List<Todo>>`를 사용합니다.

메서드	동작
<code>addTodo(String title) → bool</code>	trim한 제목이 비면 <code>false</code> , 정상 추가하면 <code>true</code>
<code>deleteTodo(int id) → void</code>	해당 ID 항목만 제외한 새 목록으로 교체
<code>toggleTodo(int id) → void</code>	해당 ID의 완료 상태를 바꾼 새 Todo-목록으로 교체

기존 목록을 직접 수정하지 않고 `List<Todo>.unmodifiable`의 새 목록을 `state`에 대입합니다. 화면은 변경을 구독하므로 결과가 바로 반영됩니다. ID는 1부터 시작하고 정상 추가마다 증가하며 삭제 후에도 실행 중 재사용하지 않습니다. 같은 제목도 ID가 다르면 별개 항목이고, 없는 ID의 삭제·완료 변경은 목록을 바꾸지 않습니다.

서버나 파일 저장을 기다리는 비동기 단계가 없으므로 동기 `Notifier`를 사용합니다. 앱 재시작 또는 새 `ProviderScope`에서는 빈 목록과 ID 1부터 시작합니다.

6. 네이티브 DevTools와 촬영

DevTools는 Flutter SDK에 포함되어 별도 설치가 필요하지 않습니다. 앱은 **Windows native 프로세스**로 실행하고 분석 도구 화면은 브라우저에서 엽니다. 연결 화면의 앱 종류와 실행 모드를 확인합니다.

```
flutter run -d windows
```

Inspector는 debug 앱의 터미널에 출력된 DevTools 링크로 연결합니다. 위젯 트리를 펼쳐 속성·부모 구조를 확인합니다. 프레임워크 위젯도 보려면 [Show Implementation Widgets](#)를 클릭합니다. 캡처에서는 Column을 선택해 vertical 방향, start 정렬, stretch 교차 정렬과 padding 16을 확인했습니다.

debug 실행을 **q**로 종료한 뒤 profile 앱을 실행하고 이번 실행의 DevTools 링크에 연결합니다.

```
flutter run -d windows --profile
```

Timeline·Memory·Performance는 **Windows profile 앱**에서 확인합니다. 해당 실행의 DevTools에 연결하여 항목 추가·체크·삭제·스크롤을 수행합니다. DevTools 2.60.0의 **Performance → Timeline Events**에서 `todo.add/delete/toggle` 표식과 선택 구간을 확인하고, 프레임 상세에서 UI-raster 시간을 확인합니다. Memory에서는 그래프와 클래스 수를 관찰하며 필요하면 GC-Refresh 전후를 비교합니다.

실제 profile 실행에서 추가 20회·삭제 10회·완료 변경 1회 후 10개가 남았습니다. Timeline Events의 [Refresh timeline events](#)로 기록을 가져온 뒤 `todo.add`를 검색해 다음 결과를 선택했습니다. 검색 결과는 20건이고 선택 이벤트는 Category Dart, Duration **116us**였습니다.

Performance의 프레임 390을 선택했을 때 **UI 0.3ms / Raster 8.0ms / Paint 0.1ms**, **Raster Jank Detected**가 표시됐습니다. Memory에서 GC와 Refresh 후 **Todo 10개 / TodoTile 10개**와 All Classes 행의 Dart Heap **10.6 MB**를 확인했습니다. 클래스 표는 화면의 기본 필터가 적용된 상태이고 그래프의 RSS와 Dart heap은 다른 지표입니다.

촬영용 `test_driver`는 실제 `main.dart`를 호출하면서 FlutterDriver 검증 확장과 warm-up 프레임 호출을 포함합니다. 따라서 차트의 평균 FPS를 일반 사용의 평균 성능으로 주장하지 않습니다. 한 프레임 시간은 전체 평균이 아니며 메모리 값에는 검증 도구 비용도 포함됩니다. 짧은 GC 관찰만으로 누수가 없다고 단정하지 않습니다. 일반 관찰은 위의 기본 profile 명령으로 재현합니다.

DevTools 서버가 따로 필요하면 다른 터미널에서 실행합니다.

```
dart devtools --port 9100 --no-launch-browser
```

출력된 주소에 접속하여 현재 앱의 VM Service URL을 연결합니다. **VM 주소와 토큰은 실행마다 달라집니다.** 이전 실행의 주소를 재사용하지 않습니다. 참고: [DevTools CLI 실행](#), [Performance 사용](#).

동일한 간단 버전 1.1.0+3의 기능 화면 5장과 DevTools 화면 4장을 촬영했습니다. Readme.pdf 부록에 9장 모두 포함했습니다.

파일: docs/screenshots/	화면
01_list.png	제목·완료 체크가 보이는 목록
02_add_input.png	새 제목 입력
03_add_result.png	추가된 항목과 변경된 목록
04_delete_before.png	삭제 대상과 삭제 전 목록
05_delete_after.png	해당 항목만 삭제된 목록
06_devtools_inspector.jpg	debug 위젯 트리·선택 속성
07_devtools_timeline.jpg	profile의 Timeline Events
08_devtools_memory.jpg	profile 메모리 그래프·클래스
09_devtools_performance.jpg	profile 프레임 차트·UI/raster

실행 화면: 목록과 완료 표시 (뒤의 실제 캡처 부록 참조)

실행 화면: DevTools Timeline (뒤의 실제 캡처 부록 참조)

7. 검증 상태

간단 버전 1.1.0+3의 정적 분석은 **No issues found!**였으며 새 소스 복사본에서 의존성 설치, **14개 테스트(상태 6개·화면 8개)**와 일반 `lib/main.dart`의 **Windows release 빌드**를 통과했습니다. OneDrive의 기존 테스트 캐시 삭제 오류가 있어 새 복사본에서 검증했습니다. 실제 앱 소스 해시를 기록해 동일 소스인지 확인했습니다.

debug 앱에서 목록·추가·삭제·완료 체크를 확인했으며 항목 수는 3 → 4 → 3으로 변경됐습니다. 일반 release 실행 파일에서도 키보드 입력·Enter 추가 0 → 1개, 종료 후 재실행 시 0개 초기화를 확인했습니다. Windows profile 앱을 DevTools에 연결해 Timeline·Memory·Performance를 검사했습니다.

근거는 `docs/evidence/simple_final_validation.json`, `simple_final_test.txt`, `simple_feature_capture.json`, `simple_release_smoke.json`, `profile_workload.json`, `devtools_observations.json`입니다. VM 타임라인은 순환 버퍼이므로 이후의 읽기 결과에서는 일부 초기 이벤트가 빠질 수 있습니다.

8. PDF와 제출 코드

```
powershell -ExecutionPolicy Bypass -File .\scripts\update_readme.ps1
powershell -ExecutionPolicy Bypass -File .\scripts\package_submission.ps1
```

PDF는 `output/pdf/Readme.pdf`에 생성됩니다. 생성에는 Windows의 맑은 고딕, Python과 `reportlab-Pillow`가 필요합니다. PATH에 Python이 없다면 `update_readme.ps1`의 `-PythonPath`로 실행 파일을 지정합니다. 직접 생성할 때에는 `python -m pip install reportlab Pillow` 후 `python scripts/build_readme_pdf.py`를 실행합니다. GitHub 루트에도 완성된 `Readme.pdf`를 제공합니다.

제출 ZIP은 `submission/today_todo_날짜_시각.zip`에 생성되고 내부 최상위에 `Readme.pdf`가 있습니다. 자체 `lib/`, `test/`, `test_driver/`, `scripts/`, 문서·실제 캡처, `pubspec.yaml`·`pubspec.lock`·분석 설정·필수 플랫폼 소스를 포함합니다. `.tooling/`, SDK·다운로드 의존성·`build/`·`.dart_tool/`·`ephemeral`·Git·개인 IDE 설정은 제외합니다. Android 설정은 프로젝트 기본 골격이며 실제 빌드·실행 검증 대상은 Windows입니다. PDF의 한글·명령·표·스크린샷과 ZIP 구성을 검토했습니다. 과제 사이트 업로드는 별도로 진행해야 합니다.

실제 실행 캡처 | 리스트

Windows 네이티브 앱의 할 일 3개 목록. 강의 복습을 완료 체크하여 취소선과 체크 상태를 확인한다.

오늘 할 일

[+ 추가](#)

전체 3개

☒ 강의 복습



☐ 운동하기



☐ 과제 정리



실제 실행 캡처 | 추가 전 입력

새 제목 "README 작성"을 입력한 화면.

오늘 할 일

할 일

README 작성

+ 추가

전체 3개

☒ 강의 복습



☐ 운동하기



☐ 과제 정리



실제 실행 캡처 | 추가 결과

README 작성 항목이 추가되어 전체 목록이 4개가 된 화면.

오늘 할 일

할 일

새 할 일을 입력하세요

+ 추가

전체 4개

☒ 강의 복습



☐ 운동하기



☐ 과제 정리



☐ README 작성



실제 실행 캡처 | 삭제 전

삭제 대상 "운동하기"가 포함된 4개 항목.

오늘 할 일

할 일

새 할 일을 입력하세요

+ 추가

전체 4개

☒ 강의 복습



☐ 운동하기



☐ 과제 정리



☐ README 작성



실제 실행 캡처 | 삭제 결과

운동하기가 삭제되고 다른 3개 항목이 남은 화면.

오늘 할 일

할 일

+ 추가

전체 3개

☒ 강의 복습



☐ 과제 정리



☐ README 작성



실제 실행 캡처 | DevTools Inspector

Windows debug 앱에서 Column을 선택해 트리와 레이아웃을 검사했다. 너비 648.0, 높이 554.7, padding 16.

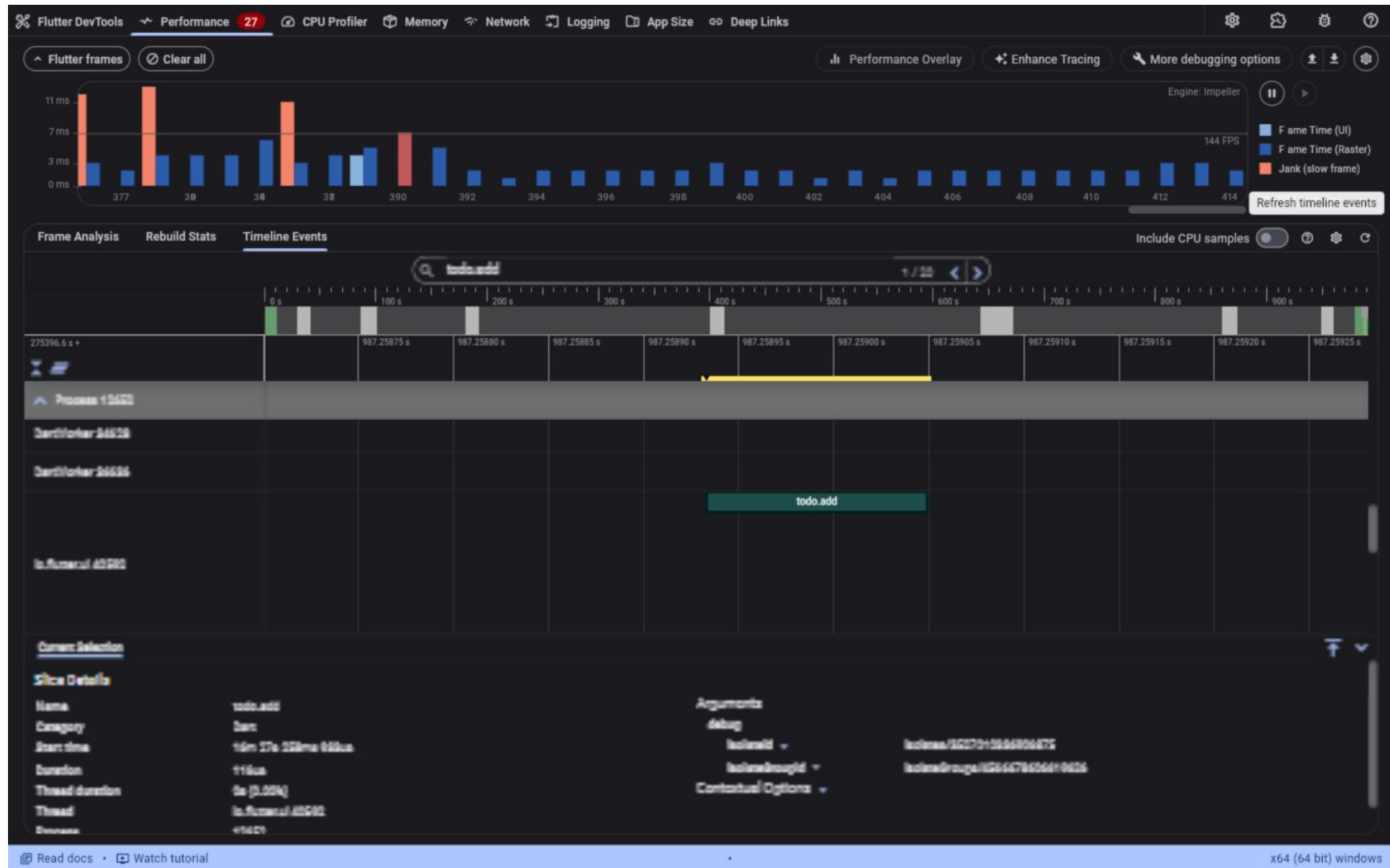
The screenshot displays the Flutter DevTools Inspector interface. The top toolbar includes various debugging tools like Performance, CPU Profiler, Memory, Debugger, Network, Logging, App Size, and Deep Links. Below the toolbar, there are tabs for 'Select Widget Mode' and 'Show Implementation Widgets'. The main area is divided into three panels:

- Widget Tree:** A hierarchical tree of widgets. The selected widget is a `Column` widget, which is a child of a `Row` widget, which is a child of a `LayoutId-<ScaffoldSlot.body>` widget, which is a child of a `CustomMultiChildLayout` widget, which is a child of a `_Theater` widget, which is a child of the `[root]` widget.
- Widget properties:** A panel showing the properties of the selected `Column` widget. It includes a visual representation of the widget's layout with dimensions and padding. The properties listed are:
 - height: 554.7
 - width: 648.0
 - top padding: 16.0
 - bottom padding: 16.0
 - left padding: 16.0
 - right padding: 16.0
 - direction: vertical
 - mainAxisAlignment: start
 - crossAxisAlignment: stretch
 - mainAxisSize: default max
 - textDirection: default null
 - verticalDirection: default down
 - textBaseline: default null
 - clipBehavior: default none
 - spacing: default 0.0
- Console:** A panel showing the console output. It displays the selected widget's path: `[root]` and `Column`. Below the path, there is a prompt: `Eval. Enter '?' for help.`

The bottom status bar shows the current window configuration: `x64 (64 bit) windows`.

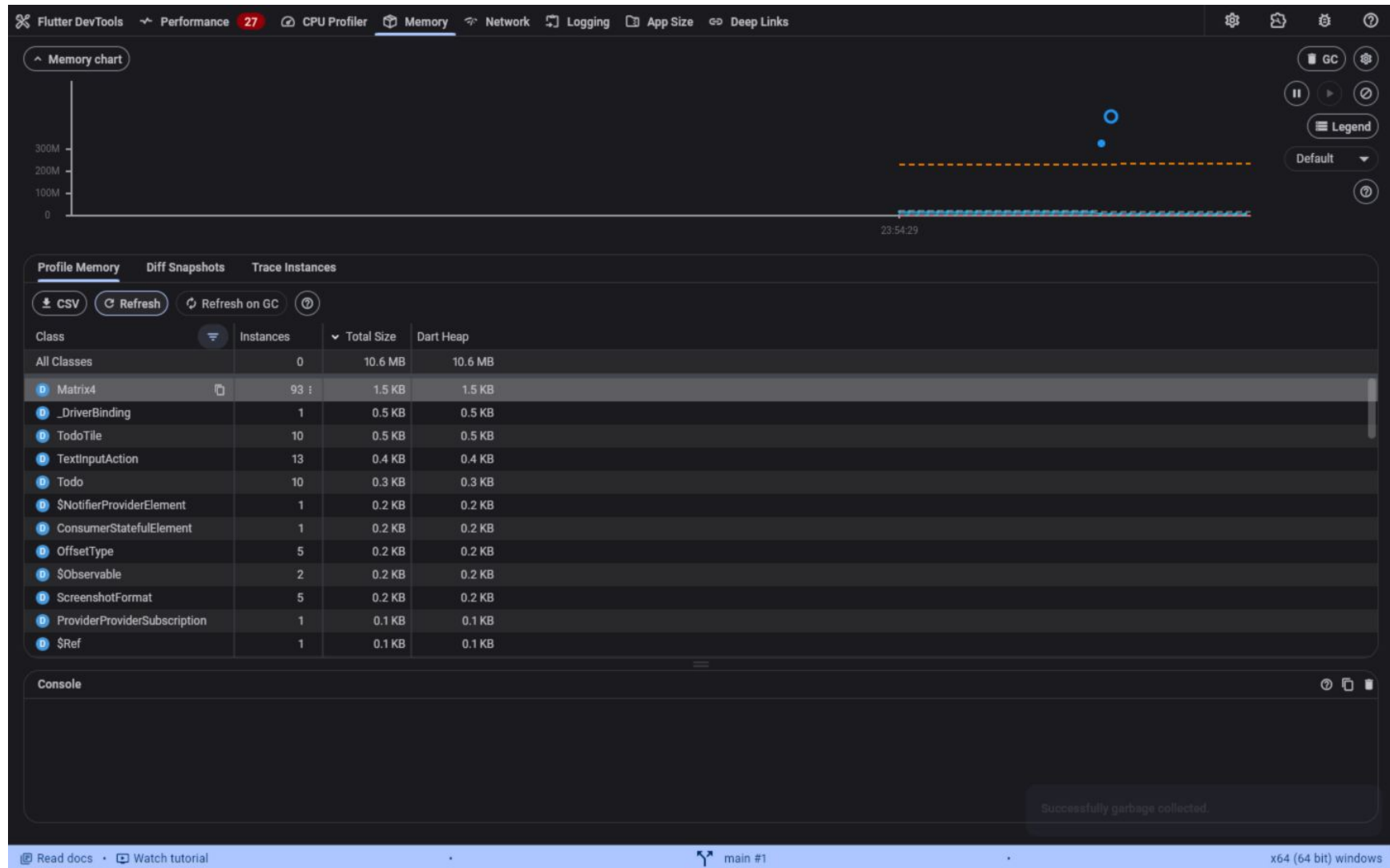
실제 실행 캡처 | DevTools Timeline

Windows profile 앱의 Performance > Timeline Events. todo.add 20건 검색 중 선택한 이벤트의 Category는 Dart, Duration은 116us다.



실제 실행 캡처 | DevTools Memory

20개 추가 후 10개 삭제, 완료 체크 1회 수행. GC와 Refresh 후 Todo 10개, TodoTile 10개를 확인했다. 짧은 관찰로 메모리 누수 여부를 단정하지 않는다.



실제 실행 캡처 | DevTools Performance

선택 프레임 390의 UI 0.3ms, Raster 8.0ms, Paint 0.1ms. Raster Jank Detected가 표시됐다. 촬영용 프레임 호출을 포함하므로 표시 FPS는 일반 사용 평균 성능으로 해석하지 않는다.

